Smart Farm Management System

A Full-Stack Java & IoT College Project

OOP II Module — Project Documentation

Covers: OOP Design · Multithreading · Networking · Database · JavaFX · IoT

Module:	Object-Oriented Programming II
Hardware:	ESP32 + DHT11 Sensor
Stack:	Java, MySQL, JavaFX, Arduino C++
Year:	2025 / 2026

Contents

1. Introduction & Project Overview

1.1 What Is This Project?

The **Smart Farm Management System** is a complete, real-world application that manages every aspect of running a farm — from monitoring field conditions with physical sensors to tracking crops, assigning workers to tasks, logging harvests, and generating reports. It combines everything studied in the OOP II module into one coherent system.

The word "smart" is not decoration. The system does not just *display* data — it *acts* on it. When the temperature in a field crosses a dangerous threshold, the system automatically generates an alert, creates a task ("irrigate Plot 3"), and assigns it to an available worker. Sensor data drives farm decisions.

The System in One Sentence

A physical sensor measures field conditions → a Java server processes the data → the system manages crops, workers, tasks, and harvests in a MySQL database → the farmer monitors and controls everything through a live JavaFX dashboard.

1.2 Core Capabilities

The system provides six integrated capabilities:

1. **Live Environmental Monitoring** — real temperature and humidity data from physical IoT sensors, displayed in real time with historical charts.
2. **Intelligent Alert System** — automatic detection of dangerous conditions with severity levels that trigger actionable responses.
3. **Crop & Plot Management** — full lifecycle tracking of crops from planting through growth stages to harvest, tied to specific farm plots.
4. **Worker & Task Management** — assignment of workers to tasks on specific plots, with status tracking and workload balancing.
5. **Harvest Logging & Analytics** — recording of yield quantity and quality grades, with comparison against expected output.
6. **Report Export** — CSV export of sensor logs, harvest summaries, and alert histories for external analysis.

1.3 Why Agriculture?

Agriculture is an ideal domain because it demands every OOP II concept simultaneously. Sensor data arrives continuously and from multiple sources, requiring **multithreading**. Different crops have different needs, requiring **inheritance and polymorphism**. Data must be persisted, requiring a **database**. The farmer needs a clear interface, requiring a **GUI**. And the physical sensor adds **networking** and **IoT** integration.

1.4 OOP II Concepts Covered

Concept	How It Appears in This Project	Module Topic
OOP Design	`Crop`, `Plot`, `Worker`, `Alert` class hierarchies	Classes & Inheritance
Interfaces	`Runnable`, `DAOInterface`, `Exportable`	Abstraction
Multithreading	One thread per ESP32 device, alert checker thread	Concurrency
Networking	TCP socket between ESP32 and Java server	Java Sockets
Database (JDBC)	MySQL storage of all readings, crops, alerts, tasks	JDBC
JavaFX GUI	Live dashboard with charts, tables, forms, alerts	GUI Programming
File I/O	CSV export of sensor logs and harvest reports	I/O Streams
Collections	`List`, `Map`, `Queue` for managing data	Java Collections
Exception Handling	Network drops, DB errors, invalid sensor data	Error Handling

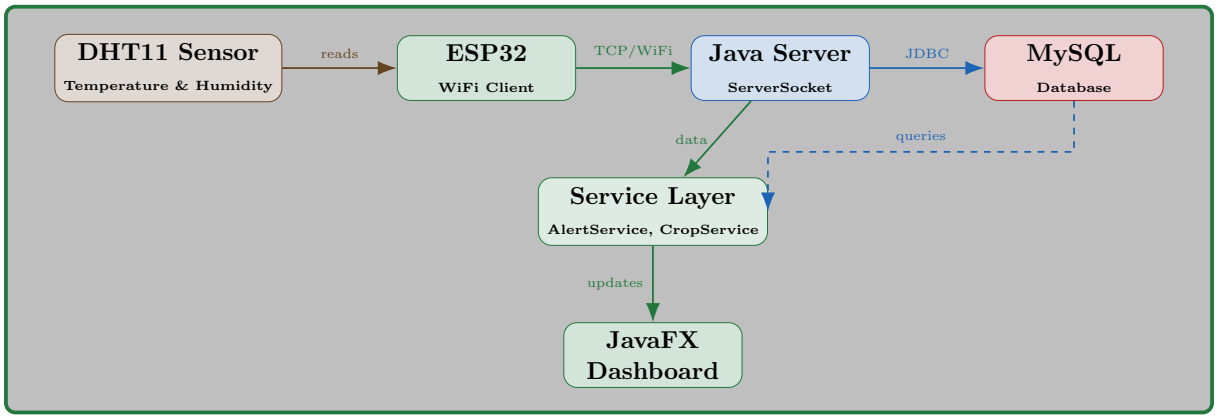
Table 1.1: OOP II concepts and their role in the project

2. System Architecture

2.1 High-Level Overview

The system consists of three physical components and four software layers:

1. **The ESP32 Device** — a microcontroller with a DHT11 sensor. Reads temperature and humidity, sends data over WiFi.
2. **The Java Application** — the server, business logic, and JavaFX dashboard all run here. It receives sensor data, manages farm operations, and presents the UI.
3. **The MySQL Database** — persistent storage for all data: readings, crops, plots, workers, tasks, alerts, and harvests.



2.2 Layered Architecture

The software follows a strict layered architecture where each layer only communicates with the layer directly below it.

Layer	What Lives Here	Technology
Presentation Layer	JavaFX screens, FXML files, UI controllers	JavaFX / FXML
Service Layer	Business logic: alerts, crop lifecycle, task assignment, harvest analytics	Plain Java classes
Data Access Layer	All database queries (DAO classes)	JDBC + MySQL
Infrastructure Layer	Sockets, DB connection pool, ESP32 link	Java Sockets

Table 2.1: The four layers and what lives in each one

The rule: the Presentation layer never writes SQL. The DAO layer never decides whether to fire an alert. Each layer has one job.

2.2.1 MVC in the JavaFX UI

Inside the JavaFX layer, Model-View-Controller is used:

- **Model** — plain Java classes like `Crop`, `Worker`, `Alert` (data only)
- **View** — `.fxml` files defining layout (no logic)
- **Controller** — Java classes connecting the View to the Service layer

2.2.2 Package Structure

```
smartfarm/
+-- model/      ->  Crop.java, Plot.java, Worker.java, Task.java,
|                  Alert.java, SensorReading.java, HarvestRecord.java
+-- dao/        ->  CropDAO.java, PlotDAO.java, SensorDAO.java,
|                  AlertDAO.java, WorkerDAO.java, TaskDAO.java,
|                  HarvestDAO.java
+-- server/     ->  FarmServer.java, SensorHandler.java
+-- service/    ->  AlertService.java, SensorService.java,
|                  CropService.java, TaskService.java,
|                  HarvestService.java, WorkerService.java
+-- ui/         ->  DashboardController.java, CropController.java,
|                  WorkerController.java, AlertController.java,
|                  HarvestController.java
|                  dashboard.fxml, crops.fxml, workers.fxml,
|                  alerts.fxml, harvest.fxml
+-- util/       ->  DBConnection.java, CSVExporter.java,
|                  ThresholdConfig.java
\-- Main.java   ->  Entry point (starts server + opens dashboard)
```

3. The OOP Model — Classes & Relationships

This chapter defines every model class in the system. These are the building blocks that every other layer depends on.

3.1 Plot

A `Plot` represents a physical piece of land on the farm.

```
public class Plot {
    private int    plotId;
    private String name;
    private String location;
    private double sizeAcres;
    private String soilType;

    public Plot(String name, String location, double sizeAcres,
        String soilType) {
        this.name      = name;
        this.location  = location;
        this.sizeAcres = sizeAcres;
        this.soilType  = soilType;
    }

    // Getters and setters omitted for brevity

    @Override
    public String toString() {
        return String.format("Plot[%s] %s     %.1f acres, %s soil
            ",
                plotId, name, sizeAcres, soilType);
    }
}
```

Listing 3.1: Plot model class

3.2 Crop

A `Crop` is planted on a `Plot` and moves through a defined lifecycle.

```
public class Crop {
    public enum GrowthStage { PLANTED, GROWING, READY, HARVESTED
        }

```

```
    private int          cropId;
    private String       cropName;
    private LocalDate    plantingDate;
    private LocalDate    harvestDate;
    private GrowthStage  growthStage;
    private int          plotId;
    private double       expectedYieldKg;

    public Crop(String cropName, LocalDate plantingDate,
        LocalDate harvestDate,
                int plotId, double expectedYieldKg) {
        this.cropName        = cropName;
        this.plantingDate    = plantingDate;
        this.harvestDate     = harvestDate;
        this.growthStage     = GrowthStage.PLANTED;
        this.plotId          = plotId;
        this.expectedYieldKg = expectedYieldKg;
    }

    /** Advance to next growth stage */
    public void advanceStage() {
        switch (growthStage) {
            case PLANTED  -> growthStage = GrowthStage.GROWING;
            case GROWING  -> growthStage = GrowthStage.READY;
            case READY    -> growthStage = GrowthStage.HARVESTED;
            case HARVESTED -> { /* already done */ }
        }
    }

    /** Check if crop is overdue for harvest */
    public boolean isOverdue() {
        return growthStage == GrowthStage.READY
            && LocalDate.now().isAfter(harvestDate);
    }

    // Getters and setters...
}
```

Listing 3.2: Crop model class with lifecycle enum

3.3 Worker and Task

A `Worker` is a farm employee. A `Task` is a job assigned to a worker on a specific plot.

```
public class Worker {
    private int    workerId;
    private String name;
    private String role;
    private String phone;
    private int    activeTaskCount; // derived from task table
```

```
7
8       public Worker(String name, String role, String phone) {
9           this.name  = name;
10          this.role  = role;
11          this.phone = phone;
12      }
13
14      /** Check if worker is available (has fewer than 3 active
            tasks) */
15      public boolean isAvailable() {
16          return activeTaskCount < 3;
17      }
18
19      // Getters and setters...
20  }
```

Listing 3.3: Worker model class

```
1   public class Task {
2       public enum Status { PENDING, IN_PROGRESS, DONE }
3
4       private int        taskId;
5       private String     description;
6       private Status     status;
7       private LocalDate  dueDate;
8       private int        workerId;
9       private int        plotId;
10      private String     alertType; // null if manually created, set
            if auto-generated
11
12      public Task(String description, LocalDate dueDate, int
            workerId, int plotId) {
13          this.description = description;
14          this.dueDate     = dueDate;
15          this.workerId    = workerId;
16          this.plotId      = plotId;
17          this.status      = Status.PENDING;
18      }
19
20      public void advanceStatus() {
21          switch (status) {
22              case PENDING     -> status = Status.IN_PROGRESS;
23              case IN_PROGRESS -> status = Status.DONE;
24              case DONE        -> { /* already complete */ }
25          }
26      }
27
28      public boolean isOverdue() {
29          return status != Status.DONE && LocalDate.now().isAfter(
                dueDate);
30      }
```

```
31
32      // Getters and setters...
33  }
```

Listing 3.4: Task model class with status lifecycle

3.4 SensorReading

```
1  public class SensorReading {
2      private int           readingId;
3      private String        deviceId;
4      private float         temperature;
5      private float         humidity;
6      private LocalDateTime timestamp;
7
8      public SensorReading(String deviceId, float temperature,
9                           float humidity, LocalDateTime timestamp)
                                {
10         this.deviceId    = deviceId;
11         this.temperature = temperature;
12         this.humidity    = humidity;
13         this.timestamp   = timestamp;
14     }
15
16     @Override
17     public String toString() {
18         return String.format("[%s] Device: %s | Temp: %.1f  C  |
               Hum: %.1f%%",
19                 timestamp, deviceId, temperature, humidity);
20     }
21 }
```

Listing 3.5: SensorReading model class

3.5 Alert

```
1  public class Alert {
2      public enum Severity { INFO, WARNING, CRITICAL }
3
4      private int           alertId;
5      private String        alertType;
6      private Severity      severity;
7      private String        message;
```

```
8       private boolean       resolved;
9       private LocalDateTime timestamp;
10      private int           plotId;
11
12      public Alert(String alertType, Severity severity, String
            message, int plotId) {
13          this.alertType = alertType;
14          this.severity  = severity;
15          this.message   = message;
16          this.plotId    = plotId;
17          this.resolved  = false;
18          this.timestamp = LocalDateTime.now();
19      }
20
21      public void resolve() {
22          this.resolved = true;
23      }
24  }
```

Listing 3.6: Alert model class

3.6 HarvestRecord

```
1   public class HarvestRecord {
2       public enum Grade { A, B, C }
3
4       private int       recordId;
5       private LocalDate harvestDate;
6       private double    quantityKg;
7       private Grade     grade;
8       private int       cropId;
9
10      public HarvestRecord(LocalDate harvestDate, double quantityKg
            ,
11                           Grade grade, int cropId) {
12          this.harvestDate = harvestDate;
13          this.quantityKg  = quantityKg;
14          this.grade       = grade;
15          this.cropId      = cropId;
16      }
17  }
```

Listing 3.7: HarvestRecord model class

3.7 Class Relationship Summary

Relationship	Description
Plot has many Crops	One plot can grow multiple crops over time
Plot has many Alerts	Alerts are tied to a specific plot's sensor
Plot has many Tasks	Work on a plot generates tasks
Worker has many Tasks	A worker can be assigned multiple tasks
Task belongs to Plot	Every task is performed on a specific plot
Crop has many Harvests	One crop can be harvested in batches
Alert can create Task	A critical alert auto-generates a task

Table 3.1: Class relationships in the system

4. The Database Layer — MySQL & JDBC

4.1 Database Schema

Every piece of data is stored in MySQL. Nothing is lost when the application closes.

```sql
-- Plots of land on the farm
CREATE TABLE plots (
    plot_id     INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    location    VARCHAR(200),
    size_acres DOUBLE,
    soil_type   VARCHAR(50)
);

-- Crops planted on each plot
CREATE TABLE crops (
    crop_id          INT PRIMARY KEY AUTO_INCREMENT,
    crop_name        VARCHAR(100) NOT NULL,
    planting_date    DATE,
    harvest_date     DATE,
    growth_stage     ENUM('PLANTED','GROWING','READY','HARVESTED'
        ),
    expected_yield   DOUBLE,
    plot_id          INT,
    FOREIGN KEY (plot_id) REFERENCES plots(plot_id)
);

-- Every reading sent by an ESP32 device
CREATE TABLE sensor_readings (
    reading_id  INT PRIMARY KEY AUTO_INCREMENT,
    device_id   VARCHAR(100) NOT NULL,
    temperature FLOAT,
    humidity    FLOAT,
    timestamp   DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Alerts generated when thresholds are crossed
CREATE TABLE alerts (
    alert_id   INT PRIMARY KEY AUTO_INCREMENT,
    alert_type VARCHAR(50),
    severity   ENUM('INFO','WARNING','CRITICAL'),
    message    TEXT,
    resolved   BOOLEAN DEFAULT FALSE,
    timestamp  DATETIME DEFAULT CURRENT_TIMESTAMP,
    plot_id    INT,
    FOREIGN KEY (plot_id) REFERENCES plots(plot_id)
);

-- Farm workers
CREATE TABLE workers (
    worker_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    name        VARCHAR(100),
    role        VARCHAR(100),
    phone       VARCHAR(20)
);

-- Tasks assigned to workers
CREATE TABLE tasks (
    task_id     INT PRIMARY KEY AUTO_INCREMENT,
    description TEXT,
    status      ENUM('PENDING','IN_PROGRESS','DONE'),
    due_date    DATE,
    alert_type  VARCHAR(50), -- NULL if manual, set if auto-
        generated
    worker_id   INT,
    plot_id     INT,
    FOREIGN KEY (worker_id) REFERENCES workers(worker_id),
    FOREIGN KEY (plot_id)   REFERENCES plots(plot_id)
);

-- Harvest events
CREATE TABLE harvest_records (
    record_id    INT PRIMARY KEY AUTO_INCREMENT,
    harvest_date DATE,
    quantity_kg  DOUBLE,
    grade        ENUM('A','B','C'),
    crop_id      INT,
    FOREIGN KEY (crop_id) REFERENCES crops(crop_id)
);
```

Listing 4.1: MySQL database schema

4.2 DAO Pattern

Each table has a corresponding DAO (Data Access Object) class. The DAO's only job is to talk to the database — it contains no business logic. Below are two representative DAOs to show the pattern.

```
public class SensorDAO {
    private final Connection conn = DBConnection.getInstance();

    public void saveReading(SensorReading reading) throws
        SQLException {
        String sql = "INSERT INTO sensor_readings (device_id,
            temperature, "
                + "humidity, timestamp) VALUES (?, ?, ?, ?)";
        try (PreparedStatement stmt = conn.prepareStatement(sql))
            {
            stmt.setString(1, reading.getDeviceId());
            stmt.setFloat (2, reading.getTemperature());
```

```
            stmt.setFloat (3, reading.getHumidity());
            stmt.setObject(4, reading.getTimestamp());
            stmt.executeUpdate();
        }
    }

    public List<SensorReading> getLast50Readings(String deviceId)
            throws SQLException {
        List<SensorReading> list = new ArrayList<>();
        String sql = "SELECT * FROM sensor_readings WHERE
            device_id = ? "
                + "ORDER BY timestamp DESC LIMIT 50";
        try (PreparedStatement stmt = conn.prepareStatement(sql))
            {
            stmt.setString(1, deviceId);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                list.add(new SensorReading(
                    rs.getString("device_id"),
                    rs.getFloat("temperature"),
                    rs.getFloat("humidity"),
                    rs.getObject("timestamp", LocalDateTime.class
                        )
                ));
            }
        }
        return list;
    }
}
```

Listing 4.2: SensorDAO — saves and retrieves sensor readings

```
public class CropDAO {
    private final Connection conn = DBConnection.getInstance();

    public void addCrop(Crop crop) throws SQLException {
        String sql = "INSERT INTO crops (crop_name, planting_date
            , harvest_date, "
                + "growth_stage, expected_yield, plot_id) "
                + "VALUES (?, ?, ?, ?, ?, ?)";
        try (PreparedStatement stmt = conn.prepareStatement(sql))
            {
            stmt.setString(1, crop.getCropName());
            stmt.setDate  (2, Date.valueOf(crop.getPlantingDate()
                ));
            stmt.setDate  (3, Date.valueOf(crop.getHarvestDate())
                );
            stmt.setString(4, crop.getGrowthStage().name());
            stmt.setDouble(5, crop.getExpectedYieldKg());
            stmt.setInt   (6, crop.getPlotId());
            stmt.executeUpdate();
```

```
16          }
17      }
18
19      public void updateGrowthStage(int cropId, Crop.GrowthStage
            stage)
20              throws SQLException {
21          String sql = "UPDATE crops SET growth_stage = ? WHERE
                crop_id = ?";
22          try (PreparedStatement stmt = conn.prepareStatement(sql))
                {
23              stmt.setString(1, stage.name());
24              stmt.setInt    (2, cropId);
25              stmt.executeUpdate();
26          }
27      }
28
29      public List<Crop> getCropsByPlot(int plotId) throws
            SQLException {
30          List<Crop> list = new ArrayList<>();
31          String sql = "SELECT * FROM crops WHERE plot_id = ? "
32                     + "ORDER BY planting_date DESC";
33          try (PreparedStatement stmt = conn.prepareStatement(sql))
                {
34              stmt.setInt(1, plotId);
35              ResultSet rs = stmt.executeQuery();
36              while (rs.next()) {
37                  Crop c = new Crop(
38                      rs.getString("crop_name"),
39                      rs.getDate("planting_date").toLocalDate(),
40                      rs.getDate("harvest_date").toLocalDate(),
41                      rs.getInt("plot_id"),
42                      rs.getDouble("expected_yield")
43                  );
44                  c.setCropId(rs.getInt("crop_id"));
45                  // Set growth stage from DB value
46                  c.setGrowthStage(Crop.GrowthStage.valueOf(
47                      rs.getString("growth_stage")));
48                  list.add(c);
49              }
50          }
51          return list;
52      }
53
54      public List<Crop> getOverdueCrops() throws SQLException {
55          List<Crop> list = new ArrayList<>();
56          String sql = "SELECT * FROM crops WHERE growth_stage = '
                READY' "
57                     + "AND harvest_date < CURDATE()";
58          try (PreparedStatement stmt = conn.prepareStatement(sql))
                {
59              ResultSet rs = stmt.executeQuery();
60              while (rs.next()) {
61                  // same mapping as above...
62              }
63          }
```

```
64          return list;
65      }
66  }
```

Listing 4.3: CropDAO — full CRUD for crops

```
1   public class TaskDAO {
2       private final Connection conn = DBConnection.getInstance();
3
4       public void addTask(Task task) throws SQLException {
5           String sql = "INSERT INTO tasks (description, status,
                due_date, "
6                   + "alert_type, worker_id, plot_id) "
7                   + "VALUES (?, ?, ?, ?, ?, ?)";
8           try (PreparedStatement stmt = conn.prepareStatement(sql))
                {
9               stmt.setString(1, task.getDescription());
10              stmt.setString(2, task.getStatus().name());
11              stmt.setDate  (3, Date.valueOf(task.getDueDate()));
12              stmt.setString(4, task.getAlertType());
13              stmt.setInt   (5, task.getWorkerId());
14              stmt.setInt   (6, task.getPlotId());
15              stmt.executeUpdate();
16          }
17      }
18
19      public int getActiveTaskCount(int workerId) throws
            SQLException {
20          String sql = "SELECT COUNT(*) FROM tasks "
21                  + "WHERE worker_id = ? AND status != 'DONE'";
22          try (PreparedStatement stmt = conn.prepareStatement(sql))
                {
23              stmt.setInt(1, workerId);
24              ResultSet rs = stmt.executeQuery();
25              rs.next();
26              return rs.getInt(1);
27          }
28      }
29
30      public List<Task> getOverdueTasks() throws SQLException {
31          List<Task> list = new ArrayList<>();
32          String sql = "SELECT * FROM tasks WHERE status != 'DONE'
                "
33                  + "AND due_date < CURDATE()";
34          try (PreparedStatement stmt = conn.prepareStatement(sql))
                {
35              ResultSet rs = stmt.executeQuery();
36              while (rs.next()) {
37                  Task t = new Task(
38                      rs.getString("description"),
39                      rs.getDate("due_date").toLocalDate(),
```

```
                        rs.getInt("worker_id"),
                        rs.getInt("plot_id")
                    );
                    t.setTaskId(rs.getInt("task_id"));
                    t.setStatus(Task.Status.valueOf(rs.getString("
                        status")));
                    list.add(t);
                }
            }
            return list;
        }
    }
```

Listing 4.4: TaskDAO — task queries including workload counting

5. The IoT Data Pipeline — ESP32, Networking & Server

This chapter covers the entire path data takes from the physical sensor to the Java application. It is one pipeline, presented as one chapter.

5.1 The DHT11 Sensor

The DHT11 is a small sensor (roughly $2) that measures temperature (0–50°C, ±2°C accuracy) and humidity (20–80% RH, ±5% accuracy). It connects to the ESP32 with three wires: power (3.3V), ground, and data (GPIO 4).

5.2 What the ESP32 Does

The ESP32 runs a simple loop: connect to WiFi, connect to the Java server via TCP, read the DHT11 every 5 seconds, format the data as text, and send it. If the connection drops, it retries automatically.

The data format is a single text line:

```
DEVICE:plot1_sensor,TEMP:27.50,HUM:63.20
```

5.3 ESP32 Firmware

```
#include <WiFi.h>
#include <DHT.h>

const char* WIFI_SSID     = "YourWiFiName";
const char* WIFI_PASSWORD = "YourWiFiPassword";
const char* SERVER_IP     = "192.168.1.100";
const int   SERVER_PORT   = 8080;
const char* DEVICE_ID     = "plot1_sensor";

#define DHT_PIN  4
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);
WiFiClient client;

void setup() {
    Serial.begin(115200);
    dht.begin();
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
```

```
    }
    Serial.println("\nWiFi connected: " + WiFi.localIP().toString
        ());
}

void loop() {
    if (!client.connected()) {
        if (!client.connect(SERVER_IP, SERVER_PORT)) {
            delay(5000);
            return;
        }
    }

    float temperature = dht.readTemperature();
    float humidity    = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
        delay(2000);
        return;
    }

    String data = "DEVICE:" + String(DEVICE_ID)
                + ",TEMP:"  + String(temperature, 2)
                + ",HUM:"   + String(humidity, 2);
    client.println(data);
    delay(5000);
}
```

Listing 5.1: ESP32 firmware — sends DHT11 data to Java server

5.4 Java Server — Receiving Connections

The Java server uses `ServerSocket` to listen for ESP32 connections. Each connected device gets its own handler thread via an `ExecutorService`, so multiple sensors can connect simultaneously.

```
public class FarmServer {
    private static final int PORT = 8080;
    private ServerSocket serverSocket;
    private ExecutorService threadPool;

    public void start() {
        threadPool = Executors.newCachedThreadPool();
        try {
            serverSocket = new ServerSocket(PORT);
            System.out.println("Farm Server started on port " +
                PORT);

            while (true) {
```

```
                Socket deviceSocket = serverSocket.accept();
                System.out.println("Device connected: "
                    + deviceSocket.getInetAddress());
                threadPool.submit(new SensorHandler(deviceSocket)
                    );
            }
        } catch (IOException e) {
            System.err.println("Server error: " + e.getMessage())
                ;
        }
    }
}
```

Listing 5.2: FarmServer — listens for ESP32 connections

5.5 SensorHandler — Per-Device Thread

```
public class SensorHandler implements Runnable {
    private final Socket socket;
    private final SensorService sensorService = new SensorService
        ();

    public SensorHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream()))
                    {
            String line;
            while ((line = reader.readLine()) != null) {
                SensorReading reading = parseReading(line);
                if (reading != null) {
                    sensorService.processReading(reading);
                }
            }
        } catch (IOException e) {
            System.out.println("Device disconnected: "
                + socket.getInetAddress());
        }
    }

    private SensorReading parseReading(String raw) {
        try {
            String[] parts = raw.split(",");
            String deviceId = parts[0].split(":")[1];
            float  temp     = Float.parseFloat(parts[1].split(":"
```

```
                )[1]);
            float  hum      = Float.parseFloat(parts[2].split(":"
                )[1]);
            return new SensorReading(deviceId, temp, hum,
                                     LocalDateTime.now());
        } catch (Exception e) {
            System.err.println("Bad data format: " + raw);
            return null;
        }
    }
}
```

Listing 5.3: SensorHandler — one thread per connected device

6. The Service Layer — Business Logic

The service layer is where the system makes decisions. This is the "brain" of the Smart Farm. Each service class handles one domain of logic and uses the DAO layer to persist results.

6.1 SensorService — Processing Incoming Data

When a reading arrives from the ESP32, `SensorService` coordinates the response: save it, check for alerts, and notify the UI.

```
public class SensorService {
    private final SensorDAO  sensorDAO  = new SensorDAO();
    private final AlertService alertService = new AlertService();

    public void processReading(SensorReading reading) {
        try {
            // 1. Save to database
            sensorDAO.saveReading(reading);

            // 2. Determine which plot this sensor belongs to
            int plotId = resolvePlotId(reading.getDeviceId());

            // 3. Check for dangerous conditions
            alertService.checkAndAlert(reading, plotId);

            // 4. Notify the JavaFX dashboard (thread-safe)
            Platform.runLater(() ->
                DashboardController.getInstance().
                    refreshSensorData()
            );
        } catch (SQLException e) {
            System.err.println("Failed to process reading: "
                + e.getMessage());
        }
    }

    private int resolvePlotId(String deviceId) {
        // Maps device IDs to plot IDs (e.g., "plot1_sensor" ->
            1)
        return Integer.parseInt(
            deviceId.replaceAll("[^0-9]", ""));
    }
}
```

Listing 6.1: SensorService — coordinates processing of each reading

6.2 AlertService — Intelligent Threshold Detection

Every time a sensor reading arrives, `AlertService` checks it against configurable thresholds. When a threshold is breached, it creates an alert **and** auto-generates a task assigned to the least-busy available worker.

Measurement	Threshold	Severity	Triggered Action
Temperature	> 35°C	CRITICAL	Alert + auto-task: "Irrigate plot"
Temperature	> 30°C	WARNING	Alert only
Temperature	< 5°C	CRITICAL	Alert + auto-task: "Cover crops"
Humidity	$< 30\%$	WARNING	Alert + auto-task: "Start irrigation"
Humidity	$> 85\%$	WARNING	Alert only (fungal risk notice)

Table 6.1: Alert thresholds and their triggered actions

```
1  public class AlertService {
2      private final AlertDAO  alertDAO  = new AlertDAO();
3      private final TaskService taskService = new TaskService();
4
5      public void checkAndAlert(SensorReading reading, int plotId)
           {
6          float temp = reading.getTemperature();
7          float hum  = reading.getHumidity();
8
9          //        Temperature checks
10         if (temp > 35.0f) {
11             createAlertWithTask("HIGH_TEMP", "CRITICAL",
12                 "Extreme heat: " + temp + "C - crops may wilt!",
13                 plotId, "Irrigate plot immediately");
14         } else if (temp > 30.0f) {
15             createAlertOnly("HIGH_TEMP", "WARNING",
16                 "High temperature: " + temp + "C", plotId);
17         } else if (temp < 5.0f) {
18             createAlertWithTask("FROST_RISK", "CRITICAL",
19                 "Frost risk: " + temp + "C - protect crops!",
20                 plotId, "Cover crops with frost protection");
21         }
22
23         //        Humidity checks
24         if (hum < 30.0f) {
25             createAlertWithTask("LOW_HUMIDITY", "WARNING",
26                 "Low humidity: " + hum + "% - irrigation needed",
27                 plotId, "Start irrigation system");
28         } else if (hum > 85.0f) {
```

```
29                createAlertOnly("HIGH_HUMIDITY", "WARNING",
30                    "High humidity: " + hum + "% - fungal risk",
31                    plotId);
32            }
33        }
34
35        /** Creates alert AND auto-generates a task for it */
36        private void createAlertWithTask(String type, String severity
              ,
37                String msg, int plotId, String taskDescription) {
38            createAlertOnly(type, severity, msg, plotId);
39            taskService.autoCreateTask(taskDescription, plotId, type)
                  ;
40        }
41
42        private void createAlertOnly(String type, String severity,
43                String msg, int plotId) {
44            Alert alert = new Alert(type,
45                Alert.Severity.valueOf(severity), msg, plotId);
46            try {
47                alertDAO.save(alert);
48                System.out.println("[ALERT] " + severity + ": " + msg
                      );
49            } catch (SQLException e) {
50                System.err.println("Failed to save alert: "
51                    + e.getMessage());
52            }
53        }
54    }
```

Listing 6.2: AlertService — checks thresholds, fires alerts, creates tasks

6.3 TaskService — Automatic Task Assignment

`TaskService` handles both manually created tasks and auto-generated tasks from alerts. For auto-tasks, it finds the least-busy available worker.

```
1   public class TaskService {
2       private final TaskDAO   taskDAO   = new TaskDAO();
3       private final WorkerDAO workerDAO = new WorkerDAO();
4
5       /** Manually assign a task to a specific worker */
6       public void createTask(String description, LocalDate dueDate,
7                              int workerId, int plotId) {
8           Task task = new Task(description, dueDate, workerId,
                plotId);
9           try {
10              taskDAO.addTask(task);
11          } catch (SQLException e) {
```

```
12              System.err.println("Failed to create task: "
13                  + e.getMessage());
14          }
15      }
16
17      /** Auto-create a task from an alert     finds the best
            worker */
18      public void autoCreateTask(String description, int plotId,
19                                 String alertType) {
20          try {
21              Worker worker = findLeastBusyWorker();
22              if (worker == null) {
23                  System.out.println("No available workers for auto
                        -task!");
24                  return;
25              }
26
27              Task task = new Task(description,
28                  LocalDate.now().plusDays(1), // due tomorrow
29                  worker.getWorkerId(), plotId);
30              task.setAlertType(alertType);
31              taskDAO.addTask(task);
32
33              System.out.println("[AUTO-TASK] '" + description
34                  + "' assigned to " + worker.getName());
35          } catch (SQLException e) {
36              System.err.println("Failed to auto-create task: "
37                  + e.getMessage());
38          }
39      }
40
41      /** Find the worker with the fewest active tasks */
42      private Worker findLeastBusyWorker() throws SQLException {
43          List<Worker> workers = workerDAO.getAllWorkers();
44          Worker bestWorker = null;
45          int minTasks = Integer.MAX_VALUE;
46
47          for (Worker w : workers) {
48              int count = taskDAO.getActiveTaskCount(w.getWorkerId
                    ());
49              if (count < minTasks && count < 3) { // max 3 active
                    tasks
50                  minTasks = count;
51                  bestWorker = w;
52              }
53          }
54          return bestWorker;
55      }
56
57      /** Update task status */
58      public void advanceTaskStatus(int taskId) {
59          try {
60              taskDAO.advanceStatus(taskId);
61          } catch (SQLException e) {
62              System.err.println("Failed to update task: "
```

```
                    + e.getMessage());
            }
        }
    }
```

Listing 6.3: TaskService — assigns tasks to workers intelligently

6.4 CropService — Crop Lifecycle Management

`CropService` manages the full lifecycle of a crop and detects overdue harvests.

```
public class CropService {
    private final CropDAO    cropDAO    = new CropDAO();
    private final AlertDAO   alertDAO   = new AlertDAO();

    /** Plant a new crop on a specific plot */
    public void plantCrop(String name, LocalDate plantDate,
            LocalDate harvestDate, int plotId, double
                expectedYieldKg) {
        Crop crop = new Crop(name, plantDate, harvestDate,
                             plotId, expectedYieldKg);
        try {
            cropDAO.addCrop(crop);
        } catch (SQLException e) {
            System.err.println("Failed to plant crop: "
                + e.getMessage());
        }
    }

    /** Advance a crop to its next growth stage */
    public void advanceCropStage(int cropId) {
        try {
            Crop crop = cropDAO.getCropById(cropId);
            crop.advanceStage();
            cropDAO.updateGrowthStage(cropId, crop.getGrowthStage
                ());
        } catch (SQLException e) {
            System.err.println("Failed to update crop stage: "
                + e.getMessage());
        }
    }

    /** Check all crops for overdue harvests and generate alerts
       */
    public void checkOverdueCrops() {
        try {
            List<Crop> overdue = cropDAO.getOverdueCrops();
            for (Crop c : overdue) {
                long daysLate = ChronoUnit.DAYS.between(
                    c.getHarvestDate(), LocalDate.now());
```

```
                    Alert a = new Alert("OVERDUE_HARVEST",
                        Alert.Severity.WARNING,
                        c.getCropName() + " on plot " + c.getPlotId()
                        + " is " + daysLate + " days overdue for
                            harvest",
                        c.getPlotId());
                    alertDAO.save(a);
                }
            } catch (SQLException e) {
                System.err.println("Overdue check failed: "
                    + e.getMessage());
            }
        }
    }
```

Listing 6.4: CropService — lifecycle management and overdue detection

6.5 HarvestService — Yield Logging and Analytics

```
public class HarvestService {
    private final HarvestDAO harvestDAO = new HarvestDAO();
    private final CropDAO    cropDAO    = new CropDAO();

    /** Record a harvest event */
    public void recordHarvest(int cropId, double quantityKg,
                              HarvestRecord.Grade grade) {
        HarvestRecord record = new HarvestRecord(
            LocalDate.now(), quantityKg, grade, cropId);
        try {
            harvestDAO.save(record);
            // Mark the crop as harvested
            cropDAO.updateGrowthStage(cropId,
                Crop.GrowthStage.HARVESTED);
        } catch (SQLException e) {
            System.err.println("Failed to record harvest: "
                + e.getMessage());
        }
    }

    /** Get yield summary for a plot: total kg, grade
        distribution */
    public Map<String, Object> getPlotYieldSummary(int plotId) {
        Map<String, Object> summary = new HashMap<>();
        try {
            List<HarvestRecord> records =
                harvestDAO.getByPlot(plotId);
            double totalKg = records.stream()
                .mapToDouble(HarvestRecord::getQuantityKg).sum();
            long gradeA = records.stream()
```

```
                .filter(r -> r.getGrade() == HarvestRecord.Grade.A)
                .count();
            long gradeB = records.stream()
                .filter(r -> r.getGrade() == HarvestRecord.Grade.B)
                .count();
            long gradeC = records.stream()
                .filter(r -> r.getGrade() == HarvestRecord.Grade.C)
                .count();

            summary.put("totalKg", totalKg);
            summary.put("gradeA", gradeA);
            summary.put("gradeB", gradeB);
            summary.put("gradeC", gradeC);
            summary.put("harvestCount", records.size());
        } catch (SQLException e) {
            System.err.println("Failed to get summary: "
                + e.getMessage());
        }
        return summary;
    }

    /** Compare actual yield vs expected yield */
    public double getYieldEfficiency(int cropId) {
        try {
            Crop crop = cropDAO.getCropById(cropId);
            double actualTotal = harvestDAO.getTotalYield(cropId);
            if (crop.getExpectedYieldKg() > 0) {
                return (actualTotal / crop.getExpectedYieldKg())
                    * 100;
            }
        } catch (SQLException e) {
            System.err.println("Yield calculation failed: "
                + e.getMessage());
        }
        return 0.0;
    }
}
```

Listing 6.5: HarvestService — records harvests and computes yield analytics

7. The JavaFX Dashboard

The dashboard is the farmer's single point of control for the entire system. It uses JavaFX with FXML layout files and controller classes.

7.1 Dashboard Layout

The main dashboard screen is divided into four panels:

1. **Live Sensor Panel** — current temperature and humidity with a real-time line chart updated every 5 seconds.
2. **Alert Panel** — active alerts colour-coded by severity (green for INFO, orange for WARNING, red for CRITICAL) with a "Resolve" button.
3. **Farm Overview** — summary cards showing total plots, active crops, pending tasks, and unresolved alerts.
4. **Navigation Sidebar** — links to Crops, Workers, Tasks, Harvest, and Export screens.

7.2 Live Chart — Thread-Safe UI Updates

Because sensor data arrives on the server thread, UI updates must be marshalled to the JavaFX Application Thread using `Platform.runLater()`.

```
public class DashboardController {
    @FXML private LineChart<String, Number> tempChart;
    @FXML private LineChart<String, Number> humChart;
    @FXML private Label currentTemp;
    @FXML private Label currentHum;
    @FXML private TableView<Alert> alertTable;
    @FXML private Label totalPlots;
    @FXML private Label activeCrops;
    @FXML private Label pendingTasks;

    private XYChart.Series<String, Number> tempSeries;
    private XYChart.Series<String, Number> humSeries;
    private final SensorDAO sensorDAO = new SensorDAO();
    private static DashboardController instance;

    public static DashboardController getInstance() {
        return instance;
    }

    @FXML
    public void initialize() {
        instance = this;
        tempSeries = new XYChart.Series<>();
        tempSeries.setName("Temperature");
        tempChart.getData().add(tempSeries);
```

```
26
27          humSeries = new XYChart.Series<>();
28          humSeries.setName("Humidity");
29          humChart.getData().add(humSeries);
30
31          refreshSensorData();
32          refreshOverview();
33      }
34
35      /** Called from SensorService via Platform.runLater() */
36      public void refreshSensorData() {
37          try {
38              List<SensorReading> readings =
39                  sensorDAO.getLast50Readings("plot1_sensor");
40              if (!readings.isEmpty()) {
41                  SensorReading latest = readings.get(0);
42                  currentTemp.setText(
43                      String.format("%.1f C", latest.getTemperature
                            ()));
44                  currentHum.setText(
45                      String.format("%.1f%%", latest.getHumidity())
                            );
46              }
47
48              // Update chart series
49              tempSeries.getData().clear();
50              humSeries.getData().clear();
51              DateTimeFormatter fmt =
52                  DateTimeFormatter.ofPattern("HH:mm:ss");
53              for (SensorReading r : readings) {
54                  String time = r.getTimestamp().format(fmt);
55                  tempSeries.getData().add(
56                      new XYChart.Data<>(time, r.getTemperature()))
                            ;
57                  humSeries.getData().add(
58                      new XYChart.Data<>(time, r.getHumidity()));
59              }
60          } catch (SQLException e) {
61              System.err.println("Failed to refresh sensor data");
62          }
63      }
64  }
```

Listing 7.1: DashboardController — live chart updates

7.3 Alert Panel with Resolve Action

```
public class AlertController {
    @FXML private TableView<Alert> alertTable;
    @FXML private TableColumn<Alert, String> severityCol;
    @FXML private TableColumn<Alert, String> messageCol;
    @FXML private TableColumn<Alert, String> timeCol;

    private final AlertDAO alertDAO = new AlertDAO();

    @FXML
    public void initialize() {
        severityCol.setCellValueFactory(
            data -> new SimpleStringProperty(
                data.getValue().getSeverity().name()));
        messageCol.setCellValueFactory(
            data -> new SimpleStringProperty(
                data.getValue().getMessage()));
        timeCol.setCellValueFactory(
            data -> new SimpleStringProperty(
                data.getValue().getTimestamp()
                    .format(DateTimeFormatter.ofPattern("dd/MM HH
                        :mm"))));

        // Colour rows by severity
        alertTable.setRowFactory(tv -> new TableRow<>() {
            @Override
            protected void updateItem(Alert alert, boolean empty)
                {
                super.updateItem(alert, empty);
                if (alert == null || empty) {
                    setStyle("");
                } else {
                    switch (alert.getSeverity()) {
                        case CRITICAL -> setStyle(
                            "-fx-background-color: #ffcccc;");
                        case WARNING  -> setStyle(
                            "-fx-background-color: #fff3cd;");
                        case INFO     -> setStyle(
                            "-fx-background-color: #d4edda;");
                    }
                }
            }
        });
        refreshAlerts();
    }

    @FXML
    private void onResolveClicked() {
        Alert selected = alertTable.getSelectionModel()
                                   .getSelectedItem();
        if (selected != null) {
            try {
                alertDAO.markResolved(selected.getAlertId());
```

```
                refreshAlerts();
            } catch (SQLException e) {
                System.err.println("Failed to resolve alert");
            }
        }
    }

    private void refreshAlerts() {
        try {
            List<Alert> alerts = alertDAO.getUnresolved();
            alertTable.setItems(
                FXCollections.observableArrayList(alerts));
        } catch (SQLException e) {
            System.err.println("Failed to load alerts");
        }
    }
}
```

Listing 7.2: AlertController — display and resolve alerts

8. CSV Export — File I/O

The system can export data to CSV files for external analysis (e.g. in Excel). The `CSVExporter` utility handles formatting and writing.

```
public class CSVExporter {

    /** Export sensor readings to CSV */
    public static void exportSensorData(List<SensorReading>
        readings,
                                        String filePath) throws
                                            IOException {
        try (BufferedWriter writer = new BufferedWriter(
                new FileWriter(filePath))) {
            // Header
            writer.write("Timestamp,DeviceID,Temperature,Humidity
                ");
            writer.newLine();

            // Data rows
            DateTimeFormatter fmt =
                DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"
                    );
            for (SensorReading r : readings) {
                writer.write(String.format("%s,%s,%.2f,%.2f",
                    r.getTimestamp().format(fmt),
                    r.getDeviceId(),
                    r.getTemperature(),
                    r.getHumidity()));
                writer.newLine();
            }
        }
    }

    /** Export harvest records to CSV */
    public static void exportHarvestData(List<HarvestRecord>
        records,
                                         String filePath) throws
                                             IOException {
        try (BufferedWriter writer = new BufferedWriter(
                new FileWriter(filePath))) {
            writer.write("HarvestDate,CropID,QuantityKg,Grade");
            writer.newLine();

            for (HarvestRecord r : records) {
                writer.write(String.format("%s,%d,%.2f,%s",
                    r.getHarvestDate(),
                    r.getCropId(),
                    r.getQuantityKg(),
                    r.getGrade().name()));
                writer.newLine();
            }
```

```
            }
        }
}
```

Listing 8.1: CSVExporter — generic CSV export utility

9. Build Roadmap

9.1 Suggested Build Order

Build the system in stages so that something is always working and testable at each step.

Stage 1 — Foundation (Day 1)

- Set up the MySQL database and run the schema script
- Create `DBConnection.java` utility (Singleton pattern)
- Write all model classes: `Plot`, `Crop`, `Worker`, `Task`, `Alert`, `SensorReading`, `HarvestRecord`
- Write all DAO classes and test each one

Stage 2 — Farm Management Core (Day 2-3)

- Build `CropService` and `TaskService` with business logic
- Build Crop & Plot management UI screens in JavaFX — full CRUD working
- Build Worker & Task management UI with status advancement
- Connect all screens to the service layer

Stage 3 — Networking & IoT Pipeline (Day 4)

- Build `FarmServer.java` and `SensorHandler.java`
- Test with a simulated Java client (no ESP32 yet)
- Verify data flows: client → server → database

Stage 4 — ESP32 Integration (Day 4)

- Wire the DHT11 to the ESP32 (3 wires)
- Upload the firmware with your WiFi and server IP
- Confirm real readings appear in the database

Stage 5 — Alert System & Auto-Tasks (Day 5)

- Implement `AlertService` with threshold checking
- Implement auto-task creation from critical alerts
- Connect alerts to the dashboard UI with colour-coded severity

Stage 6 — Full Dashboard (Day 6)

- Build the live sensor chart using `Platform.runLater()`
- Build the alert panel with resolve button
- Build the farm overview cards (summary statistics)
- Connect all screens with navigation sidebar

Stage 7 — Harvest, Export & Polish (Day 7)

- Build Harvest logging screen with yield analytics
- Implement CSV export for sensor and harvest data
- Add error handling for network drops and DB failures
- Implement overdue crop detection
- Write final documentation

10. Technology Stack & Hardware Reference

10.1 Technology Stack

Component	Technology	Purpose
Language	Java 17+	Main application language
GUI	JavaFX	Dashboard and all UI screens
Database	MySQL 8	Persistent storage
DB Driver	JDBC (mysql-connector-j)	Java to MySQL communication
Server	Java ServerSocket	Receives ESP32 connections
Concurrency	ExecutorService	Thread pool for device handlers
Microcontroller	ESP32	Field device with WiFi
Sensor	DHT11	Measures temperature & humidity
Firmware	Arduino C++	ESP32 programming language
IDE (Java)	IntelliJ IDEA / Eclipse	Java development
IDE (ESP32)	Arduino IDE	ESP32 firmware development
Build Tool	Maven or Gradle	Java dependency management

Table 10.1: Full technology stack

10.2 Hardware Shopping List

Component	Description	Price (approx)
ESP32 Dev Board	Microcontroller with built-in WiFi	$5–10
DHT11 Sensor	Temperature & humidity sensor	$1–2
Breadboard	For prototyping connections	$2–3
Jumper Wires	Male-to-male, assorted	$2
USB Cable	Micro-USB for ESP32 (often included)	—
Total		**$10–17**

Table 10.2: Full hardware shopping list

10.3 DHT11 to ESP32 Wiring

DHT11 Pin	ESP32 Pin	Description
VCC (Pin 1)	3.3V	Power
Data (Pin 2)	GPIO 4	Data signal
GND (Pin 4)	GND	Ground

Table 10.3: DHT11 to ESP32 wiring (only 3 wires needed)

Important Note

The DHT11 module (on a small PCB with 3 pins) has the pull-up resistor built in. If you have the raw sensor (4 pins), add a 10kΩ resistor between VCC and the Data pin.

11. Conclusion

The Smart Farm Management System demonstrates mastery of every topic in the OOP II module — not as isolated features, but as an integrated system where each part depends on the others.

The sensor is not the project — it is the starting point. The real value is what the system *does* with the data: it detects dangerous conditions, automatically creates tasks, assigns them to workers, tracks crop lifecycles, logs harvests, computes yield efficiency, and exports reports. Every OOP II concept has a clear, meaningful role:

- **OOP design** — seven model classes with enums, lifecycle methods, and clear relationships.
- **Multithreading** — one thread per connected sensor device.
- **Networking** — TCP socket between the ESP32 and Java server.
- **JDBC and MySQL** — seven tables storing every piece of farm data.
- **Business logic** — five service classes that make intelligent decisions.
- **JavaFX** — a live dashboard with charts, tables, alerts, and CRUD screens.
- **File I/O** — CSV export for external analysis.

The demo moment ties it all together: hold the DHT11 sensor, breathe on it, and watch the dashboard chart spike, a humidity alert fire, an auto-task get assigned to a worker, and the task appear on the task board — all in real time, end to end.

Getting Started

1. Set up MySQL and run the schema script from Chapter 4.
2. Create the Java project with the package structure from Chapter 2.
3. Write all model classes (Chapter 3) and test the DAO layer first.
4. Order the ESP32 and DHT11 (1–2 weeks to arrive).
5. Build the service layer and management screens while waiting for hardware.
6. Integrate the sensor last — everything else should already work.